

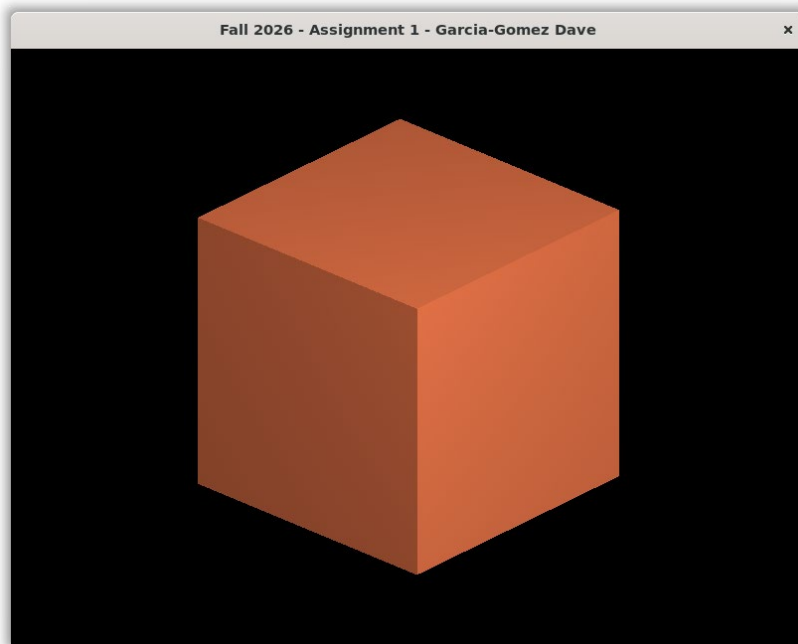
## CPSC 484 - Principles of Computer Graphics

### Assignment 1: OpenGL Mesh Viewer

Fall 2026

Professor: Dave Garcia-Gomez

Version: 2026-09-02



## 1 Getting Started

This assignment is different from Assignment 0 in one important way: nothing is filled in for you. Assignment 0's demo (the letter-viewer program, and its line-by-line commented main.cpp file) walked you through every single piece of a working OpenGL application, from window creation, shader compilation, VAO/VBO setup, uniforms, the render loop, keyboard input. This assignment does not repeat that walkthrough. Instead, you'll apply what that demo taught you to build your own program, from scratch, that displays and controls a shaded 3D cube.

# Assignment 1

## Before you start:

Before you write a single line of your own code, make sure you can explain to yourself, every piece of the Assignment 0 demo: why **GLFW** window hints have to be set before `glfwCreateWindow()`, what a **VAO** records versus what a **VBO** holds, why `glVertexAttribPointer`'s stride/offset have to match your vertex layout exactly, why `compileShader()` checks **GL\_COMPILE\_STATUS**, and how a uniform gets from a CPU-side variable into a running shader. If any of those feel shaky, go back to that demo until they don't. This assignment assumes you already own that material.

You will use the code in the **assignment\_1.tar.gz** file located in your Canvas course. Unlike Assignment 0, this is a starter, not a demo: it gives you a window that opens, GLAD loaded, an empty render loop, and the shader compile/link utility functions — the parts that are identical in every OpenGL program and that you already built once, with full understanding, in Assignment 0. It does not give you a mesh, shaders, lighting, colors, or input handling. Those are what you're building. Every place you need to add code is marked with a **"// TODO (2.x)"** comment referencing the requirement section below.

To compile the starter as-is (it will open a plain dark window and do nothing else — that's expected, and confirms your toolchain works), type **"make"** on the CPSC 484 environment or your own configured Linux/WSL/macOS machine. This should produce an executable called **assignment\_1**. If there are error messages, contact the professor.

Run the included solution application by typing **"./assignment\_1\_solution"** at the terminal, or double-clicking the compiled executable. You'll see a shaded cube sitting in the middle of an OpenGL window. Press any of the arrow keys to spin it around the x- and y-axis. Press I, J, K, and L to move the light source along the x- and y-axis, and U and O to move it along the z-axis. Press C to cycle the cube's color, and V to have it automatically transition through the range of colors. Your job is to build your own program that provides this same functionality — not to fill in blanks in a program that already provides it.

Sections of these instructions include:

1. Getting Started
2. Requirements
3. Extra Credit
4. Submission

## 2 Requirements

This is your grading breakdown for the base assignment:

### 2.1 Display Header Change (5% of grade)

Declare your own window-title string (the starter does not provide one for you. See the TODO near the top of main.cpp) and set it to "Fall 2026 - Assignment 1 - <Your Full Name>", replacing <Your Full Name> with your actual full name. Pass it into `glfwCreateWindow()` in place of the starter's placeholder "Assignment 1" literal.

### 2.2 Create the Cube Mesh (15% of grade)

Build your cube mesh entirely from scratch: decide on your vertex data (positions and normals), your index data, upload both to the GPU, and describe the layout to OpenGL yourself with `glGenVertexArrays` / `glGenBuffers` / `glBufferData` / `glVertexAttribPointer`, the same sequence the Assignment 0 demo used for its letter geometry, applied to a cube instead. The starter's TODO (2.2) marks where this setup belongs, before the render loop.

A reasonable vertex layout interleaves position and normal per vertex, the same shape the Assignment 0 demo used:

```
float vertices[] = {  
    // Positions    // Normals  
    -0.5f, -0.5f, -0.5f,  0.0f, 0.0f, -1.0f,  
     0.5f, -0.5f, -0.5f,  0.0f, 0.0f, -1.0f,  
    // ...  
};
```

When a face shares a vertex position with a neighboring face but needs a different normal, repeat that vertex position with the new normal rather than trying to reuse the same entry. Normals are used on a per-face basis, so a cube's eight corners need more than eight vertex entries once every face is accounted for. Work out why before you write the array; you'll be asked about it in your README.

Define your faces as triangles in a separate index array, referencing positions in your vertices array by index:

```
unsigned int indices[] = {  
    0, 1, 2, 2, 3, 0,  
    // ...  
};
```

# Assignment 1

---

*Triangles need to be wound counter-clockwise as seen from outside the cube, or your faces will render back-to-front.*

This setup defines two triangle faces for one side based on the indices of the vertices defined in the vertices array. Note that faces need to be defined in forms of triangles, since that's how your graphics card will optimally draw them, and each triangle's vertices would have to be defined in a counter clockwise manner to ensure the face of the triable appears in the right direction.

## 2.3 Write & Compile Your Shaders (15% of grade)

Unlike Assignment 0, your shader source is not given. Write your own vertex and fragment shaders as GLSL raw-string literals (see the TODO (2.3) markers in the starter, and the Assignment 0 demo's R"GLSL(...)GLSL" pattern) and pass them into the **createShaderProgram()** utility function the starter provides. That function itself is the same one from Assignment 0 and does not need to change.

At minimum, your vertex shader needs a position attribute, a normal attribute, and a transform uniform (mat4) so Section 2.7's rotation can move the mesh. Your fragment shader needs a color uniform (vec3) for Section 2.5, and enough of a lighting calculation, even a simple ambient + diffuse term using the light position/direction from Section 2.6, that the cube's faces are visibly shaded rather than flat silhouettes, the way the solution application's screenshot above looks.

Compile with **createShaderProgram()** and check the console the first time you run — a shader that fails to compile still returns an ID and won't crash your program, it'll just silently draw nothing.

## 2.4 Input Handler (15% of grade)

Implement your own `key_callback` (the starter declares the function and registers it with **glfwSetKeyCallback**, but its body is empty - see TODO (2.4)) to accept keyboard input. Add the keys you want the callback to capture and the action you want performed. For example:

```
if (key == GLFW_KEY_ESCAPE) {  
  
    glfwSetWindowShouldClose(window, true); // Close window on ESC  
  
}
```

# Assignment 1

---

This captures a press of the Escape key and calls `glfwSetWindowShouldClose` to close the application. Sections 2.5 through 2.7 will each add more keys to this same callback.

## 2.5 Color Changes (15% of grade)

Add the ability to change the color of the displayed cube. Declare your own state for the current color and a set of preset colors to cycle through (the starter does not predefine `cubeColor` or `presetColors` - see TODO (2.4/2.5/2.6)). Wire the **C** key to advance to the next color in your set each time it's pressed, and upload the current color to your shader's color uniform each frame. Include a minimum of 8 colors in your preset set.

## 2.6 Light Position Changes (15% of grade)

Add the ability to move the light. Declare your own light-position state, initialized to something like (1.0, 1.0, 5.0), and wire the **I**, **J**, **K**, **L**, **U**, and **O** keys to change it: **J/L** should change the x-axis value by 0.5 in each direction, **I/K** should do the same for y, and **U/O** should do the same for z. Upload the updated light position to your shader each frame so Section 2.3's lighting calculation reflects the change.

## 2.7 Mesh Rotation (20% of grade)

Once the above is working, add rotation. Declare your own rotation-angle state (e.g. `rotationX` and `rotationY`) and wire the arrow keys to change them. Use `glm::rotate` to build a rotation into your model/transform matrix each frame from these angles, and upload that matrix to your shader's transform uniform, the same way the Assignment 0 demo built and uploaded its own transform matrix for each spinning letter.

## 3 Extra Credit

### 3.1 Smooth Color Transition (additional 5% of max grade)

Add a key handler for **V** that enables the cube's color to automatically transition smoothly from one color to another, rather than jumping discretely on each **C** press. You can transition through your **presetColors** set or through a different set of colors entirely. Run the solution application and press **V** to see the target behavior.

### 3.2 Camera Motion Input Handler (additional 10% of max grade)

Add a mouse input handler that lets the camera rotate around its center origin while the left mouse button is held and the mouse moves. Holding the right mouse button and moving the mouse should pan the camera up/down and side to side. The scroll wheel should zoom the camera in and out.

## 4 Submission

### Before you start:

Make sure your code compiles. If the submitted code does not compile, that will be an automatic 25% deduction of your final assignment grade. If you develop on a different environment or platform, double check that your code compiles on your CPSC 484 environment to ensure it compiles properly there.

As a final step, write a README.txt that answers the following questions:

- Name of student:
- Did you collaborate with anyone in the class? If so, let us know who you talked to and what sort of help you gave or received.
- Were there any references (books, papers, websites, etc.) that you found particularly helpful for completing your assignment? Please provide a list.
- Why does a cube need more than eight vertex entries once normals are accounted for? What did you conclude, and how did it shape your vertices array?
- Are there any known problems with your code? If so, please provide a list and, if possible, describe what you think the cause is and how you might fix them if you had more time or motivation. This is very important, as we're much more likely to assign partial credit if you help us understand what's going on.
- Got any comments about this assignment that you'd like to share? Was it too long? Too hard? Were the requirements unclear? Did building this from scratch (rather than filling in blanks) change how well you understood it? Feel free to be brutally honest; we promise we won't take it personally.

Be thorough with your answers. Missing README.txt would be an automatic deduction of 5% of max grade.

**Submit the following online in one zip file or a tar.gz file. Do not submit multiple files.**

- Your code. Enter your name in the title of the window in main.cpp. For example:

# Assignment 1

---

```
std::string titleString = "Fall 2026 - Assignment 1 - <Name>";  
// Replace <Name> with your name
```

- A compiled executable built from your code name assignment\_1.
- The aforementioned README.txt file.
- Any additional files necessary to run and compile your program (e.g. glad.c and its headers).
- Put all files in ONE zip or tar.gz file. You should only submit one compressed file with all files included. Name the archived file "CPSC 484 - Assignment 1 - <Last Name First Name>.tar.gz". Do not include the brackets < or >.

Example: CPSC 484 – Assignment 1 – Garcia-Gomez Dave.tar.gz